

Quantization, Sparsity, and Behavioral Robustness of Qwen LLMs under Edge Hardware Constraints

Nguyen Dang Khoa
github.com/Kouskous56
Vietnam

RTX 4050 6GB, native Windows

September 2026

Abstract

Deploying large language models on memory-constrained edge hardware requires aggressive compression, yet standard metrics often fail to predict how compressed models actually behave. We present a controlled empirical study of post-training quantization and unstructured pruning of Qwen2.5 models (0.5B, 1.5B, 3B) on a 6 GB laptop GPU, combining GGUF quantization, Wanda pruning, sparsity-preserving LoRA recovery, external benchmark validation, and a debiased evaluation protocol with frozen harnesses. We find that (i) Q4_K_M provides the strongest practical size–behavior trade-off among the tested quantization levels; (ii) perplexity changes do not reliably predict behavioral degradation, and this relationship depends strongly on model scale; (iii) at fixed 30% Wanda sparsity, behavioral retention increases across the three tested Qwen2.5 scales, whereas hardware-compatible 2:4 sparsity substantially degrades quality; (iv) masked, task-aligned recovery improves same-domain behavior while preserving sparsity; and (v) unstructured sparsity produces no memory saving or measurable speedup under dense execution. Our practical operating point is Qwen2.5-3B at Q4.

1 Introduction

Deploying large language models on memory-constrained edge hardware demands aggressive compression, yet standard metrics often fail to predict how compressed models actually behave. We study post-training quantization and unstructured pruning of Qwen2.5 models (0.5B, 1.5B, 3B) on a 6 GB laptop GPU, with debiased evaluation, sparsity-preserving recovery, and deployment measurement. Edge GPUs in the 4–8 GB class cannot generally keep 7B+ FP16 models fully resident in GPU memory, so practitioners rely on quantization and pruning without knowing which metric predicts survival. We run everything on one laptop to keep hardware constant and report failures alongside successes.

2 Related Work

Post-training quantization via GPTQ [10] and AWQ [3] showed that LLMs compress to 4 bits with little loss; we test whether that optimism survives behavioral (not just loss) evaluation on Qwen2.5 [5] at three scales. One-shot pruning via SparseGPT [1] and Wanda [2] motivates our sparsity axis, including the 2:4 pattern we evaluate against unstructured masks. For recovery we build on LoRA [6] with inference-time mask reapplication. Evaluation follows the EleutherAI harness [9]

(MetaBench, WikiText) and adds IFEval [8] plus GSM8K [7] generation probes, because our early results showed loss metrics missing generative failures. Our contribution is not a new compressor but the cross-scale, behavior-first comparison under fixed edge hardware.

3 Experimental Environment

All experiments run natively on Windows 11: Acer Nitro laptop, Intel Core 7 240H, 16 GB RAM, NVIDIA RTX 4050 Laptop GPU (6 GB, CUDA 12.9, sm_89). Software: Ollama 0.34.3, Python 3.11, torch 2.9.0+cu129, transformers 5.17.0, llama.cpp (CPU build 0.4.1), upstream SparseGPT ported to Qwen2, EleutherAI lm-eval 0.4.13. Run provenance is tracked through manifests and raw logs, with legacy exceptions documented explicitly. VRAM discipline (one model at a time, 1-minute keep-alive) and RAM guards precede every heavy run.

4 Quantization Study

We quantize FP16 checkpoints to GGUF with llama-quantize (same 0.4.1 binary). Table 1 shows file sizes. Q3 exhibits repetition collapse at 0.5B; at 3B, Q4 matches the official Ollama release qualitatively while Q3 loses detail. On 50 math MCQs, self-quantized 3B-Q4 scores 36/50 vs 35/50

Table 1: GGUF file sizes (GB). Q4_K_M in bold is the sweet spot.

Model	F16	Q8	Q5	Q4	Q3
0.5B	0.95	0.51	—	0.38	0.34
3B	5.90	3.10	2.10	1.80	1.50

official; on 200 permuted items, 149 vs 147 with 183/200 identical letters (91.5%) and 188/200 identical correctness (94%). **Q4_K_M is the compression sweet spot.**

5 Benchmark Failures and V2 Methodology

Our TEST20 evaluator passed wrong answers; TEST50 revealed severe positional bias: the 0.5B model answers B on 47/50 items while the answer key itself overuses B/C, inflating raw accuracy. A pipeline audit then showed the missing chat template (B-rate 1.00 $\rightarrow$ 0.28), yet rerunning with the template merely shifted collapse toward A—positional fragility is real and format-dependent. Our frozen V2 protocol therefore uses 50 questions $\times$ 4 option rotations plus free-response generation, greedy decoding, fixed context, and full raw logs; a 3-repetition audit on the llama.cpp path gives zero variance (F16 18/18/18, Q4 16/16/16); the later Transformers GPU path shows about one-item run variation, so HF single-run scores are finite-run observations rather than deterministic estimates. Rule: never

Table 2: V2 permutation results, legacy diagnostic block (bias discovery; not directly compared with frozen pruning runs).

Model	Perm	B-rate	Cons4/50	Free/50
0.5B F16	63	0.89	1	23
0.5B Q4	58	0.81	1	26
3B mine-Q4	149	0.29	24	37
3B official	147	0.27	24	36

trust raw accuracy without positional diagnostics. We prune with the Wanda score $|W|\sqrt{E[x^2]}$ per output row, calibrating on 32 WikiText sequences (deviations documented where VRAM forced 8×128). Harnesses are frozen (greedy, fixed template and context), with run provenance recorded in manifests. External validation uses lm-eval 0.4.13: MetaBench on 0.5B dense gives 0.319 overall, with permutation leaving ARC/HellaSwag scores essentially unchanged, showing that likelihood-scored multiple

choice and generative label selection expose different failure modes. Tooling battles (console encoding, batch-size OOM, NLTK data, API payload types) are documented in run manifests to keep the external block reproducible.

6 Sparsity Curves and Behavioral Divergence

Wanda pruning (WikiText calibration, per-output $|W|\sqrt{E[x^2]}$ metric) gives standardized PPL (512/256, token-weighted, no template): 0.5B: 15.11 $\rightarrow$ s30 16.31 (+7.9%); 1.5B: 9.93 $\rightarrow$ 10.58 (+6.5%); 3B: 8.76 $\rightarrow$ 9.31 (+6.3%) $\rightarrow$ s40 10.50 $\rightarrow$ s50 12.98. Naive 2:4 on 0.5B explodes to 63.92. Uniform per-layer sparsity rules out selective layer sparing as an explanation for the observed robustness differences across scales. Calibration uses 32 WikiText sequences of 512 tokens (8×128 for 3B-s40/s50 under VRAM pressure, documented as a deviation rather than hidden). Yet at 0.5B s30, standardized PPL rises only 7.9% while the canonical same-script HF free-response score falls from 16/50 to 7/50 (44% retention); in the recovery block, WikiText-LoRA improves PPL 14.23 $\rightarrow$ 12.63 (same-block only) while generation stays 9 $\rightarrow$ 9. **Perplexity alone is insufficient to characterize behavioral robustness under compression.**

Table 3: Retention at 30% sparsity (frozen harness). PPL gaps nearly identical, behavior diverges by scale; 0.5B row uses the same-harness HF pair 7/16 (perm 65/77).

Scale	Δ PPL	Free	Perm	GSM
0.5B	+7.9%	44%	84%	—
1.5B	+6.5%	76%	91%	83%
3B	+6.3%	125%	99%	92.5%

7 Masked and Domain-Matched Recovery

Plain LoRA merging destroys sparsity (0.20 $\rightarrow$ 0.00); reapplying the original mask preserves it with PPL 12.13 and generation intact (17/50 vs 18 base). On damaged s30, WikiText-LoRA fails (free 9 $\rightarrow$ 9, GSM 4 $\rightarrow$ 3) while GSM8K-matched masked LoRA restores held-out GSM8K 4 $\rightarrow$ 14 ($\times 3.5$) at 29.9% sparsity (matched model and training budget, differing recovery corpus/objective; arms

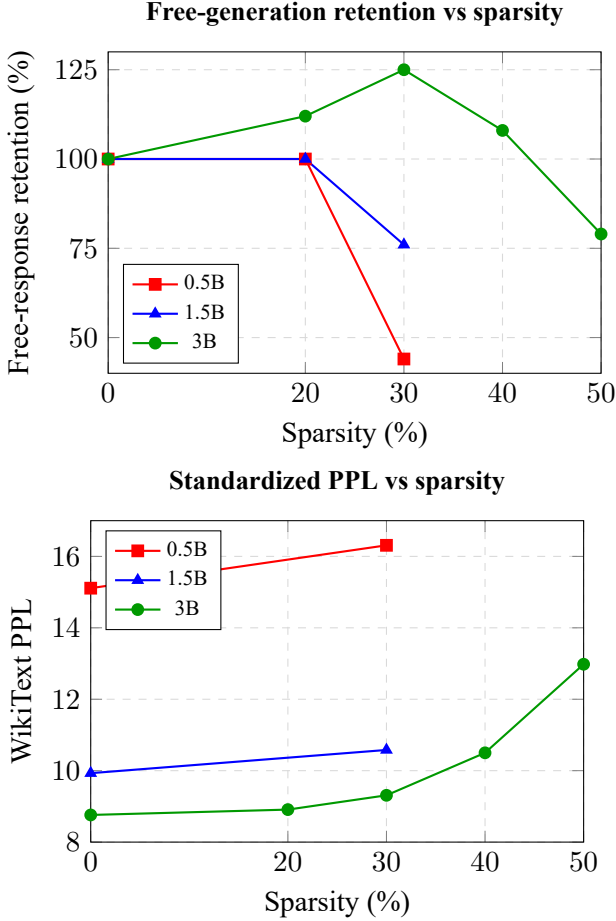


Figure 1: Capability retention (left) and standardized PPL (right) vs sparsity.

C0=4/C1=3/C2=14 on GSM-heldout). The controlled arms support task-aligned recovery data and objectives as a major driver of same-domain recovery (GSM-heldout 4 $\rightarrow$ 14 at fixed 29.9% sparsity). Masked merging did not reduce GSM8K accuracy in this run; the observed 10-vs-14 plain/masked difference is single-run variation, not evidence that masking improves accuracy. Training uses LoRA rank 8 ($\alpha=16$) on attention and MLP projections, $lr \ 2 \times 10^{-4}$, 150 steps; assistant-only loss for instruction data.

8 Scale-Dependent Robustness and the 3B Cliff

Free-generation retention at fixed 30% Wanda sparsity increased across the three tested scales (44%, 76%, 125% on our finite 50-item suite; 0.5B from the same-harness HF pair 7/16; values above 100% reflect sampling noise, not improvement from pruning). At 3B, s40 holds (free 108%, perm 91%, GSM 78%) while s50 collapses (free 79%, perm 62%, GSM 38%, PPL +48%, D-collapse): the cliff lies in (40%, 50%] under the tested 3B Wanda con-

Table 4: Ollama deployment footprint (Q4, 4K context). TTFT steady-state.

Model	VRAM	Decode	TTFT
0.5B	555 MiB	280 t/s	0.01–0.10 s
1.5B	1209 MiB	140 t/s	0.02 s
3B	2161 MiB	82 t/s	0.02–0.05 s
Qwen3-4B	3129 MiB	61 t/s	0.03 s

figuration (s40/s50 use reduced 8×128 calibration). GSM-heldout retention with the fixed boxed-aware extractor reaches 35/32/29 at 1.5B s0/s20/s30, corroborating the free-response trend on an independent axis. **Similar perplexity degradation does not imply similar downstream robustness across scales.**

9 Edge Deployment and Realized Benefit

Ollama deployment on RTX 4050: 0.5B 280 tok/s at 555 MiB; 1.5B 140 tok/s; 3B $\sim$ 80 tok/s at 2.2 GB with 20–50 ms steady TTFT (warm-up mandatory; cold start measured 5.2 s). An 8-prompt chatbot demo favors 3B mine-Q4 on all qualitative axes; 0.5B is $3 \times$ faster but showed substantially poorer Vietnamese instruction behavior in the 8-prompt demo, including a refusal-like failure on one prompt. Hybrid s30+Q4 keeps 94–97% instruction following (IFEval strict 0.601 $\rightarrow$ 0.566). Unstructured sparsity produced no memory saving or measurable speedup under dense execution; the observed increase in energy/token is reported descriptively because generation length and single-run effects remain confounded. Semi-structured 2:4 is unavailable in native Windows PyTorch (cuSPARSELt), and SparseGPT-2:4 collapses quality (PPL 15.23, free 10, GSM 2/50; ns64 only 14.80). **Quality-friendly sparsity $\neq$ hardware-friendly sparsity.**

Table 5: Dense execution realizes no memory savings from unstructured zero weights (single run; apparent decode slowdown unconfirmed).

Model	VRAM	Decode	Wall	J/tok
dense	5.8 GB	10.1	23.6 s	3.69
s30	5.8 GB	6.1	40.0 s	5.15
s50	5.8 GB	4.8	52.9 s	6.14

10 External Validation

MetaBench (lm-eval, loglikelihood) 0.5B dense: 0.319 overall; permutation leaves ARC/HellaSwag scores essentially unchanged, showing that likelihood-scored multiple choice and generative label selection expose different failure modes. IFEval on self-quantized 3B-Q4: prompt-strict 0.60, instruction-strict 0.69 (audit: 541 unique docs).

11 Conclusion

We compress Qwen2.5 models for a 6 GB edge GPU and find that Q4_K_M provides the strongest practical size–behavior trade-off among tested GGUF levels; perplexity changes do not reliably predict behavioral degradation, and this relationship depends strongly on scale; at fixed s30, free-response retention increases across the three tested scales (44%, 76%, 125% on finite suites); masked task-aligned recovery improves same-domain behavior under fixed sparsity; and unstructured sparsity produces no memory saving or measurable speedup under dense execution. The deployment sweet spot is Qwen2.5-3B at Q4. The central empirical result is the scale-dependent mismatch between perplexity degradation and behavioral degradation; the hardware experiments add a complementary mismatch between quality-friendly and hardware-friendly sparsity. For practitioners on 6 GB-class hardware, our data supports a simple recipe: quantize to Q4 first (large practical gains in our tested setup). Do not apply unstructured pruning solely for deployment efficiency unless the runtime and storage stack can exploit sparsity: on dense runtimes it saves nothing. If pruning is used, validate the chosen level behaviorally rather than relying on perplexity alone. For 3B under the tested Wanda setup, s30 is a high-retention regime, s40 shows moderate degradation, and s50 exhibits broad collapse.

12 Limitations and Future Work

Single runs in places; WikiText-32 calibration (8×128 for 3B-s40/s50); GSM subsample 50/1319; C2-plain lacks perm; no sparse kernels on native Windows (WSL2/vLLM/TensorRT-LLM track); C3 mixed-data and s40/s50 at 1.5B open. Artifacts, manifests, and raw logs are retained for reproduction and will be released upon publication. Notably, positional bias itself shrinks with scale in exploratory (non-unified-harness) observations (B-rate $0.89 \rightarrow$

$0.4 \rightarrow 0.26$); a unified-harness rerun is needed before claiming scale dependence. Single runs dominate (9A proves determinism for the llama.cpp harness only; the Transformers GPU path shows about one-item variation, so HF scores are finite-run observations); GSM uses 50-item subsamples; GSM8K recovery is evaluated on held-out samples from the same task distribution; it demonstrates in-distribution generalization, not cross-domain recovery. 2:4 stops at ns64 per a preregistered gate.

A Master Numbers

Table 6: Standardized PPL (512/256). s50-0.5B historical.

Scale	s0	s30	s40	s50
0.5B	15.11	16.31	–	22.13
1.5B	9.93	10.58	–	–
3B	8.76	9.31	10.50	12.98

Table 7: Recovery arms on s30-0.5B (GSM-heldout/50).

Arm	Sparsity	Free	GSM
C0 none	0.299	9	4
C1 WikiLoRA	0.299	9	3
C2 GSM masked	0.299	11	14

References

- [1] Frantar & Alistarh. SparseGPT. arXiv:2301.00774, 2023.
- [2] Sun et al. Wanda. arXiv:2306.11695, 2023.
- [3] Lin et al. AWQ. arXiv:2306.00978, 2023.
- [4] Kipnis et al. MetaBench. arXiv:2407.12844, 2024.
- [5] Yang et al. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.
- [6] Hu et al. LoRA. arXiv:2106.09685, 2021.
- [7] Cobbe et al. GSM8K. arXiv:2110.14168, 2021.
- [8] Zhou et al. IFEval. arXiv:2311.07911, 2023.
- [9] Gao et al. LM Evaluation Harness. Zenodo, 2023.
- [10] Frantar et al. GPTQ. arXiv:2210.17323, 2022.